PPL - Week 9

Adarsh Ranjan
230962278, AIML - A

Q1.

```
#include <stdio.h>
#include <cuda.h>

__global__ void csr_spmv(int n, int *rowPtr, int *colInd, float *val,
float *x, float *y) {
    int row = threadIdx.x + blockIdx.x * blockDim.x;

    if (row < n) {
        float sum = 0;
        for (int j = rowPtr[row]; j < rowPtr[row + 1]; j++) {
            sum += val[j] * x[colInd[j]];
        }
        y[row] = sum;
    }
}

int main() {
    int n = 3;

    int rowPtr[] = {0, 2, 4, 5};
    int colInd[] = {0, 2, 0, 1, 2};
    float val[] = {1, 2, 3, 4, 5};

    float x[] = {1, 2, 3};
    float y[3];

    int *d_rowPtr, *d_colInd;
    float *d_val, *d_x, *d_y;

    cudaMalloc(&d_rowPtr, sizeof(rowPtr));
    cudaMalloc(&d_colInd, sizeof(colInd));
    cudaMalloc(&d_val, sizeof(val));
    cudaMalloc(&d_x, sizeof(x));
    cudaMalloc(&d_y, sizeof(y));
```

```
    cudaMemcpy(d_rowPtr, rowPtr, sizeof(rowPtr), cudaMemcpyHostToDevice);
    cudaMemcpy(d_colInd, colInd, sizeof(colInd), cudaMemcpyHostToDevice);
    cudaMemcpy(d_val, val, sizeof(val), cudaMemcpyHostToDevice);
    cudaMemcpy(d_x, x, sizeof(x), cudaMemcpyHostToDevice);

    csr_spmv<<<1, n>>>(n, d_rowPtr, d_colInd, d_val, d_x, d_y);

    cudaMemcpy(y, d_y, sizeof(y), cudaMemcpyDeviceToHost);

    printf("Result:\n");
    for (int i = 0; i < n; i++)
        printf("%f ", y[i]);
    printf("\nAdarsh Ranjan, 230962278");

    cudaFree(d_rowPtr); cudaFree(d_colInd);
    cudaFree(d_val); cudaFree(d_x); cudaFree(d_y);

    return 0;
}
```

```
(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week9$ nvcc q1.cu -o q1
(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week9$ ./q1
Result:
7.000000 11.000000 15.000000
Adarsh Ranjan, 230962278(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week9$
```

Q2.

```
#include <stdio.h>
#include <cuda.h>
#include <math.h>

__global__ void transform(float *A, int m, int n) {
    int row = blockIdx.y;
    int col = threadIdx.x;

    if (row < m && col < n) {
        int idx = row * n + col;
        A[idx] = pow(A[idx], row + 1);
```

```
    }
}

int main() {
    int m = 3, n = 3;
    float A[] = {
        1, 2, 3,
        4, 5, 6,
        7, 8, 9
    };

    float *d_A;
    cudaMalloc(&d_A, sizeof(A));
    cudaMemcpy(d_A, A, sizeof(A), cudaMemcpyHostToDevice);

    dim3 grid(1, m);
    dim3 block(n);

    transform<<<grid, block>>>(d_A, m, n);

    cudaMemcpy(A, d_A, sizeof(A), cudaMemcpyDeviceToHost);

    printf("Output:\n");
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++)
            printf("%.0f ", A[i*n + j]);
        printf("\n");
    }
    printf("\nAdarsh Ranjan, 230962278");


    cudaFree(d_A);
    return 0;
}
```

```
Adarsh Ranjan, 230962278(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week9$ nvcc q2.cu -o q2
(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week9$ ./q2
Output:
1 2 3
16 25 36
343 512 729

Adarsh Ranjan, 230962278(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week9$ nvcc q3.cu -o q3
```

Q3.

```cuda
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>
#include <stdlib.h>

__global__ void complement_k(int *a, int m, int n) {
    int i = blockIdx.x;
    int j = threadIdx.x;

    if (i > 0 && i < m - 1 && j > 0 && j < n - 1) {
        int v = a[i * n + j];
        int res = 0;
        int p = 1;
        if (v == 0) {
            res = 1;
        } else {
            while (v > 0) {
                int bit = v % 2;
                int flipped = (bit == 0) ? 1 : 0;
                res = res + (flipped * p);
                p *= 10;
                v /= 2;
            }
        }
        a[i * n + j] = res;
    }
}

int main() {
    int *a, m, n;
    int *d_a;
```

```
    printf("Enter dimensions m and n: ");
    if (scanf("%d %d", &m, &n) != 2) return -1;

    int size = m * n * sizeof(int);
    a = (int*)malloc(size);

    printf("Enter Matrix A:\n");
    for (int i = 0; i < m * n; i++) {
        scanf("%d", &a[i]);
    }

    cudaMalloc((void**)&d_a, size);
    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);

    complement_k<<<m, n>>>(d_a, m, n);
    cudaMemcpy(a, d_a, size, cudaMemcpyDeviceToHost);

    printf("\nResultant Matrix B:\n");
    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            printf("%d\t", a[i * n + j]);
        }
        printf("\n");
    }
    printf("\nAdarsh Ranjan, 230962278");

    cudaFree(d_a);
    free(a);


    return 0;
}
```

```
Adarsh Ranjan, 230962278(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week9$ nvcc q3.cu -o q
(base) mca@computinglab25-22:~/Desktop/PPL_230962278/Week9$ ./q3
Enter dimensions m and n: 4 4
Enter Matrix A:
1
2
3
4
6
5
8
3
2
4
10
1
9
1
2
5

Resultant Matrix B:
1       2       3       4
6       10      111     3
2       11      101     1
9       1       2       5
```